P01-07 · T16–T17 Fixture Generator & the Full Test Suite

> A test suite that only tests the happy path is a decoration. This page builds the **hostile** corpus first — zero-byte RAWs, truncated IFDs, 4,000-character paths, emoji filenames, NFD twins, a file that grows while you read it — and only then the tests. If the fixtures are honest, the tests cannot be dishonest.

1. `cargo xtask fixtures` — the generator

Fixtures are **generated, not committed**. A 4,000-file RAW corpus is 220 GB; committing it is impossible, and downloading it makes CI depend on a network. Instead we generate byte-identical fixtures from a seed, so every machine and every CI runner produces the same corpus.

```
// xtask/src/fixtures.rs

use std::{fs, io::Write, path::{Path, PathBuf}};

/// Fixture generation is deterministic: same seed, same bytes, on ever
y OS.
/// We use a counter-based PRNG rather than `rand::thread_rng` so there
is no
/// entropy anywhere in the pipeline.
struct Prng(u64);

impl Prng {
    fn new(seed: u64) -> Self { Self(seed) }
    /// SplitMix64. Small, fast, and identical on every platform, which
is the
    /// only property we need.
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
```

```
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn next_range(&mut self, lo: u64, hi: u64) -> u64 {
        if hi <= lo { return lo; }
        lo + self.next_u64() % (hi - lo)
    }
}

#[derive(Debug, Clone, Copy)]
pub enum Corpus {
    /// 40 files. Runs in under a second. Used by every unit test.
    Tiny,
    /// 400 files, 2 GB. The PR lane. Every behaviour is represented at
least once.
    Small,
    /// 4,000 files, 22 GB of synthetic RAW-shaped data. The nightly la
ne and
    /// the budget measurements.
    Full,
    /// The hostile corpus: 60 files, every one of them a deliberate pr
oblem.
    Hostile,
}

pub fn generate(corpus: Corpus, out: &Path, seed: u64) -> anyhow::Resul
t<Manifest> {
    fs::create_dir_all(out)?;
    let mut prng = Prng::new(seed);
    let mut manifest = Manifest::default();

    match corpus {
        Corpus::Tiny   => wedding_day(out, &mut prng, &mut manifest, 4
0, 1)?,
        Corpus::Small  => wedding_day(out, &mut prng, &mut manifest, 40
0, 2)?,
        Corpus::Full   => wedding_day(out, &mut prng, &mut manifest, 40
00, 3)?,
        Corpus::Hostile => hostile(out, &mut prng, &mut manifest)?,
    }
```

```
    // The manifest records the expected outcome for every file, so tes
ts assert
    // against a declared truth rather than against whatever the code p
roduced.
    manifest.write_toml(&out.join("manifest.toml"))?;
    Ok(manifest)
}

/// A synthetic wedding day: three cameras, two photographers, realisti
c folder
/// structure, realistic burst timing, one camera with a clock one hour
off.
fn wedding_day(out: &Path, prng: &mut Prng, m: &mut Manifest, count: us
ize, cameras: u8)
    -> anyhow::Result<()>
{
    let sections = [
        ("01-preparation", 0.15), ("02-ceremony", 0.30),
        ("03-portraits", 0.20), ("04-reception", 0.25), ("05-party", 0.
10),
    ];
    let bodies: &[(&str, &str, i64)] = &[
        ("Canon",  "EOS R5",  0),        // correct clock
        ("Nikon",  "Z 9",     3600),     // one hour fast: the DST clas
sic
        ("Sony",   "ILCE-7RM5", -7),     // seven seconds slow, realist
ic drift
    ];

    let mut index = 0usize;
    for (section, share) in sections {
        let n = ((count as f64) * share) as usize;
        for i in 0..n {
            let (make, model, skew) = bodies[(i % cameras as usize).min
(bodies.len() - 1)];
            let dir = out.join("CARD-A").join(section);
            fs::create_dir_all(&dir)?;

            let stem = format!("IMG_{:05}", 1000 + index);
            // Bursts: every seventh frame starts a burst of 3 to 8 fra
mes one
```

```
            // tenth of a second apart, because burst detection in PHAS
E-08 will
            // need this and generating it now costs nothing.
            let base_ts = 1_755_000_000 + (index as i64) * 4 + skew;

            let raw = dir.join(format!("{stem}.CR2"));
            write_synthetic_raw(&raw, prng, make, model, base_ts)?;
            m.expect_photo(&raw, make, model, base_ts);

            // 30 % of frames also have a camera JPEG, as photographers
do.
            if prng.next_range(0, 100) < 30 {
                let jpg = dir.join(format!("{stem}.JPG"));
                write_synthetic_jpeg(&jpg, prng, base_ts)?;
                m.expect_companion(&raw, &jpg);
            }
            index += 1;
        }
    }

    // Realistic accidents that are NOT errors and must import cleanly:
    //   - the same card copied twice into two folders
    //   - a Lightroom preview folder that must be skipped
    //   - .DS_Store and Thumbs.db
    duplicate_folder(out, "CARD-A/02-ceremony", "BACKUP/ceremony-copy",
m)?;
    fs::create_dir_all(out.join("Lightroom Catalog Previews.lrdata"))?;
    fs::write(out.join("Lightroom Catalog Previews.lrdata/junk.dat"),
b"ignore me")?;
    fs::write(out.join("CARD-A/.DS_Store"), b"\x00\x01")?;
    fs::write(out.join("CARD-A/Thumbs.db"), b"\x00\x01")?;
    Ok(())
}

/// Synthetic RAW: a real TIFF/EXIF header followed by pseudo-random "s
ensor
/// data". Not decodable by LibRaw — deliberately. Phase 01 never decod
es, and a
/// fixture that pretends to be decodable would hide that fact until PH
ASE-02.
fn write_synthetic_raw(path: &Path, prng: &mut Prng, make: &str, model:
```

```
&str, ts: i64)
    -> anyhow::Result<()>
{
    let mut f = fs::File::create(path)?;
    f.write_all(&tiff_header_with_exif(make, model, ts))?;
    // 5 MB of incompressible bytes: enough to make hashing measurable,
small
    // enough that 4,000 of them fit in 22 GB.
    let mut buf = vec![0u8; 64 * 1024];
    for _ in 0..80 {
        for chunk in buf.chunks_mut(8) {
            chunk.copy_from_slice(&prng.next_u64().to_le_bytes()[..chun
k.len()]);
        }
        f.write_all(&buf)?;
    }
    Ok(())
}
```

2. The hostile corpus — 60 deliberate problems

```
// xtask/src/fixtures.rs (continued)

fn hostile(out: &Path, prng: &mut Prng, m: &mut Manifest) -> anyhow::Re
sult<()> {
    let d = out.join("HOSTILE");
    fs::create_dir_all(&d)?;

    // --- Empty and truncated ------------------------------------
---------
    fs::write(d.join("zero-byte.CR2"), b"")?;
    m.expect_quarantine("zero-byte.CR2", "AURA-IO-1010");

    let full = tiff_header_with_exif("Canon", "EOS R5", 1_755_000_000);
    fs::write(d.join("truncated-header.CR2"), &full[..12])?;
    m.expect_import_without_exif("truncated-header.CR2");

    fs::write(d.join("exif-says-8000-bytes-file-has-40.CR2"), lying_ifd
())?;
    m.expect_import_without_exif("exif-says-8000-bytes-file-has-40.CR
```

```
2");

    // --- Adversarial EXIF ----------------------------------------
---------
    fs::write(d.join("circular-ifd.CR2"), ifd_pointing_at_itself())?;
    m.expect_import_without_exif("circular-ifd.CR2");
    fs::write(d.join("exif-bomb-500k-tags.jpg"), exif_with_tag_count(50
0_000))?;
    m.expect_import_capped("exif-bomb-500k-tags.jpg");
    fs::write(d.join("gps-lat-91.jpg"), exif_with_gps(91.0, 10.0))?;
    m.expect_gps_dropped("gps-lat-91.jpg");
    fs::write(d.join("date-1899.jpg"), exif_with_date("1899:12:31 23:5
9:59"))?;
    m.expect_capture_time_rejected("date-1899.jpg");
    fs::write(d.join("date-2099.jpg"), exif_with_date("2099:01:01 00:0
0:00"))?;
    m.expect_capture_time_rejected("date-2099.jpg");
    fs::write(d.join("date-feb-30.jpg"), exif_with_date("2026:02:30 12:
00:00"))?;
    m.expect_capture_time_rejected("date-feb-30.jpg");
    fs::write(d.join("shutter-denominator-zero.jpg"), exif_with_rationa
l(1, 0))?;
    m.expect_import_without_shutter("shutter-denominator-zero.jpg");

    // --- Hostile names -------------------------------------------
---------
    // Unicode: NFD and NFC of the same visible name. One photo, not tw
o.
    fs::write(d.join("cafe\u{0301}-nfd.CR2"), synthetic(prng))?;   // e
+ combining acute
    fs::write(d.join("caf\u{00e9}-nfc.CR2"), synthetic(prng))?;    // p
recomposed e-acute
    m.expect_two_distinct_photos_different_bytes();

    fs::write(d.join("\u{1F389}-emoji-\u{1F4F8}.CR2"), synthetic(prn
g))?;
    fs::write(d.join("\u{0627}\u{0644}\u{0639}\u{0631}\u{0648}\u{0633}-
rtl.CR2"), synthetic(prng))?;
    fs::write(d.join("\u{4E2D}\u{6587}\u{6587}\u{4EF6}\u{540D}.CR2"), s
ynthetic(prng))?;
    fs::write(d.join("space at end .CR2"), synthetic(prng))?;
```

```
    fs::write(d.join("...leading-dots.CR2"), synthetic(prng))?;
    fs::write(d.join("semi;colon,comma'quote.CR2"), synthetic(prng))?;
    fs::write(d.join("percent%20encoded.CR2"), synthetic(prng))?;
    m.expect_all_import_cleanly();

    // Reserved Windows device names. Creating these fails on Windows,
which is
    // itself the expected behaviour, so the generator records the plat
form.
    #[cfg(not(windows))]
    {
        fs::write(d.join("CON.CR2"), synthetic(prng))?;
        fs::write(d.join("NUL.CR2"), synthetic(prng))?;
        m.expect_import_or_quarantine_by_platform();
    }

    // Deep nesting: 300 levels, then a file. Exceeds MAX_PATH without
the
    // extended prefix and exceeds our own depth limit of 32.
    let mut deep = d.join("deep");
    for i in 0..300 { deep = deep.join(format!("lvl{i:03}")); }
    if fs::create_dir_all(&deep).is_ok() {
        let _ = fs::write(deep.join("buried.CR2"), synthetic(prng));
        m.expect_skipped_by_depth_limit("deep/**/buried.CR2");
    }

    // A single 4,000-character filename component where the OS allows
it.
    let long_name = format!("{}.CR2", "x".repeat(240));
    if fs::write(d.join(&long_name), synthetic(prng)).is_ok() {
        m.expect_import_cleanly(&long_name);
    }

    // --- Structural traps ---------------------------------------
---------
    #[cfg(unix)]
    {
        // Symlink loop: must terminate, must not be followed.
        let loop_dir = d.join("loop");
        fs::create_dir_all(&loop_dir)?;
        std::os::unix::fs::symlink(&loop_dir, loop_dir.join("self"))?;
```

```
        m.expect_scan_terminates();

        // A named pipe. Opening it would block for ever.
        let _ = std::process::Command::new("mkfifo").arg(d.join("pipe.C
R2")).status();
        m.expect_ignored_not_a_regular_file("pipe.CR2");
    }

    // Two files, identical bytes, different names. One photo, two file
rows.
    let bytes = synthetic(prng);
    fs::write(d.join("twin-a.CR2"), &bytes)?;
    fs::write(d.join("twin-b.CR2"), &bytes)?;
    m.expect_same_content_hash("twin-a.CR2", "twin-b.CR2");

    // An .xmp with no image beside it: kept, never discarded.
    fs::write(d.join("orphan-sidecar.xmp"), b"<x:xmpmeta/>")?;
    m.expect_retained_unattached("orphan-sidecar.xmp");

    // A JPEG whose extension lies: .CR2 containing JPEG magic bytes.
    fs::write(d.join("actually-a-jpeg.CR2"), jpeg_magic_plus(prng))?;
    m.expect_import_kind_from_extension("actually-a-jpeg.CR2");

    // 200 MB single file: proves streaming hashing does not buffer who
le files.
    write_large(d.join("huge-200mb.CR2"), prng, 200 * 1024 * 1024)?;
    m.expect_peak_rss_under_mb("huge-200mb.CR2", 300);

    Ok(())
}
```

Category	Files	The bug it would otherwise ship
Empty and truncated	3	A panic on a slice index, or a photo silently dropped
Adversarial EXIF	7	An unbounded allocation, or a 1899 date sorting the timeline wrongly
Hostile names	11	Duplicate photos from NFD twins; a crash on emoji; SQL breakage on quotes
Path limits	3	An unhandled Windows `MAX_PATH` error mid-import
Structural traps	4	An infinite scan on a symlink loop; a permanent hang on a FIFO

Category	Files	The bug it would otherwise ship
Identity edge cases	4	Two photos for one photograph, or a discarded sidecar
Memory	1	Reading a 200 MB file into a `Vec` and blowing the RSS budget

3. Test taxonomy — what runs where

Layer	Count	Corpus	Runtime	Lane	What it proves
Unit	~160	Tiny / none	under 20 s	Every push	One function, one behaviour, no IO where avoidable
Property	14	Generated	under 60 s	Every push	Invariants over thousands of inputs, not three examples
Golden / snapshot	9	Small	under 30 s	Every push	Output is byte-stable across refactors and platforms
Integration	28	Small + Hostile	under 4 min	Every push	Crates work together against a real catalog on a real disk
Chaos	11	Small / Full	under 25 min	Nightly	The catalog survives kills, yanks, full disks and clock jumps
Performance	8	Full	under 20 min	Nightly	Every budget on the phase page is met, with numbers
Security / privacy	6	Hostile	under 30 s	Every push	No path in a log, no network call, no jargon in a message

4. Property tests — the fourteen invariants

```
// crates/aura-ingest/tests/properties.rs
use proptest::prelude::*;
```

```
proptest! {
    /// The identity invariant. If two files have the same bytes they h
ave the
    /// same hash, and if they differ they do not. Trivial to state, an
d the
    /// single most important property in the system.
    #[test]
    fn hash_is_a_function_of_content_only(
        content in prop::collection::vec(any::<u8>(), 1..100_000),
        name_a in "[a-zA-Z0-9_-]{1,40}",
        name_b in "[a-zA-Z0-9_-]{1,40}",
    ) {
        prop_assume!(name_a != name_b);
        let (ha, hb) = hash_two_files_with_same_content(&content, &name
_a, &name_b);
        prop_assert_eq!(ha, hb);
    }

    /// Import is idempotent for ANY corpus shape, not just our fixture
s.
    #[test]
    fn import_is_idempotent_for_any_file_set(files in arb_file_set(0..6
0)) {
        let cat = fresh_catalog();
        let first = import(&cat, &files);
        let digest = cat.digest();
        let second = import(&cat, &files);
        prop_assert_eq!(second.files_imported, 0);
        prop_assert_eq!(cat.digest(), digest);
    }

    /// Pairing partitions the input: every file belongs to exactly one
group,
    /// and no file is invented or lost.
    #[test]
    fn pairing_is_a_partition(files in arb_scanned_files(0..80)) {
        let groups = pair::group(&files);
        let mut seen: Vec<usize> = groups.iter()
            .flat_map(|g| std::iter::once(g.primary).chain(g.companion
s.iter().map(|(i, _)| *i)))
```

```
            .collect();
        seen.sort_unstable();
        let orphan_sidecars = count_orphan_sidecars(&files);
        prop_assert_eq!(seen.len() + orphan_sidecars, files.len());
        let before = seen.len();
        seen.dedup();
        prop_assert_eq!(seen.len(), before, "a file appeared in two gro
ups");
    }

    /// Scanning the same tree twice yields the identical order.
    #[test]
    fn scan_order_is_a_pure_function_of_the_tree(tree in arb_tree(1..4
0)) {
        let a = scan_names(&tree);
        let b = scan_names(&tree);
        prop_assert_eq!(a, b);
    }

    /// Task ids round-trip and never collide across different inputs.
    #[test]
    fn task_id_is_injective(
        stage in "[a-z]{1,12}", subject in "[a-z0-9]{1,20}", version in
1u32..99,
    ) {
        let id = TaskId::new(&stage, &subject, version).expect("vali
d");
        prop_assert_eq!(id.stage(), stage.as_str());
        prop_assert_eq!(TaskId::new(&stage, &subject, version).expect
("again"), id);
        prop_assert_ne!(TaskId::new(&stage, &subject, version + 1).expe
ct("v+1"), id);
    }

    /// Redaction never leaks a component of the original path.
    #[test]
    fn redaction_leaks_nothing(path in arb_path()) {
        let red = redact_path(&path);
        for component in path.components() {
            let s = component.as_os_str().to_string_lossy();
            if s.len() > 2 { prop_assert!(!red.contains(s.as_ref()), "l
```

```
eaked {s}"); }
        }
    }

    /// Progress is monotonic for any sequence of updates, including de
creasing ones.
    #[test]
    fn progress_never_decreases(updates in prop::collection::vec(0u64..
10_000, 1..200)) {
        let emitter = test_emitter();
        let mut last = 0;
        for u in updates { emitter.update(progress_with_done(u));
            let now = last_emitted().done;
            prop_assert!(now >= last); last = now; }
    }

    /// Every AuraError renders a user_message free of technical token
s, for
    /// every possible context we could attach.
    #[test]
    fn user_messages_stay_clean_under_any_context(ctx in "[\\PC]{0,20
0}") {
        for err in every_error_in_registry() {
            let e = err.with_context("path", &ctx);
            prop_assert!(!e.user_message.contains('/'));
            prop_assert!(!e.user_message.contains("Err"));
        }
    }

    /// The state machine never permits a terminal state to change.
    #[test]
    fn terminal_states_are_final(from in arb_terminal_state(), to in ar
b_state()) {
        prop_assert!(!from.can_transition_to(to));
    }

    /// SQL identifiers built from user data are always parameterised:
a file
    /// name containing a quote or a semicolon must never break a state
ment.
    #[test]
```

```
    fn hostile_file_names_cannot_break_sql(name in "[\\PC]{1,80}") {
        let cat = fresh_catalog();
        prop_assert!(insert_file_named(&cat, &name).is_ok());
        prop_assert_eq!(read_back_name(&cat), name);
        cat.integrity_check().expect("integrity");
    }
}
```

The remaining four properties cover typed-id parsing round-trips, `fast_key` stability under a same-size rewrite, quarantine idempotence, and dependency-promotion correctness on arbitrary DAG shapes.

5. Chaos suite — eleven scenarios

```
// tests/chaos/mod.rs

/// Each scenario is: set up, break something real, recover, assert con
vergence.
/// "Break something real" means an actual SIGKILL, an actual unmount,
an actual
/// full filesystem. A mocked failure proves only that the mock works.
pub const SCENARIOS: &[Scenario] = &[
    Scenario::KillDuringScan,
    Scenario::KillMidHashBatch,
    Scenario::KillMidInsertTransaction,
    Scenario::KillDuringWalCheckpoint,
    Scenario::KillDuringMigration,
    Scenario::KillDuringHeartbeat,
    Scenario::VolumeUnmountedAt60Percent,
    Scenario::DiskFullAt90Percent,
    Scenario::CatalogFileMadeReadOnlyMidRun,
    Scenario::WallClockJumpsBackTwoHours,
    Scenario::SecondInstanceLaunchedMidImport,
];
```

Scenario	How it is really broken	Required behaviour	Code
Kill during scan	`SIGKILL` to the child	Relaunch rescans; no partial rows; no duplicates	—
Kill mid hash batch	`SIGKILL` at file 97 of 200	At most 200 files of work lost; resume under 5 s	`AURA-JOB-7001`

Scenario	How it is really broken	Required behaviour	Code
Kill mid insert transaction	`SIGKILL` between insert and commit	Transaction rolls back whole; `integrity_check` ok	—
Kill during WAL checkpoint	`SIGKILL` while the WAL is being folded in	SQLite recovers the WAL; zero data loss beyond the last commit	—
Kill during migration	`SIGKILL` inside the migration transaction	Old schema intact; verified backup present; next launch retries	`AURA-DB-3002`
Kill during heartbeat	`SIGKILL` between claim and first heartbeat	Startup reclaim returns the task to `Ready`	`AURA-JOB-7001`
Volume unmounted at 60 %	`umount -f` / `diskutil unmount force` on a RAM disk	Run pauses, does not fail; reconnect resumes; nothing marked deleted	`AURA-IO-1003`
Disk full at 90 %	Fixed-size loopback device filled by a sibling writer	Halt with a clear message; catalog still opens; no truncated rows	`AURA-IO-1004`
Catalog made read-only mid-run	`chmod 444` on the catalog file	Run blocks with a fixable message; no corruption on restore	`AURA-IO-1002`
Wall clock jumps back 2 h	`FixedClock` wall clock moved back; monotonic untouched	No mass lease reclamation; ordering unaffected	—
Second instance mid-import	Launch a second real process	Second refuses immediately; first continues undisturbed	`AURA-DB-3005`

```
// tests/chaos/volume.rs

/// The volume-yank test, in full, because it is the one people usually
fake.
///
/// A RAM disk gives us a volume we can genuinely force-unmount without
asking
/// the developer to unplug hardware. On Linux it is a loop-mounted tmp
fs; on
/// macOS, `hdiutil attach -nomount ram://`. Windows uses a VHD. All th
ree are
/// real mounts, so the OS produces the real error we need to handle.
#[test]
#[ignore = "chaos: requires mount privileges, runs nightly"]
fn volume_disconnect_pauses_and_resumes_without_data_loss() {
    let disk = RamDisk::create_mb(4096).expect("ram disk");
```

```
    generate_fixture(Corpus::Small, disk.path(), 2).expect("fixture");

    let cat = open_catalog_on_internal_disk();
    let handle = start_import_async(&cat, disk.path());
    wait_until_progress_at_least(&handle, 0.60);

    disk.force_unmount().expect("unmount");
    let outcome = handle.join().expect("import thread must not panic");

    // The run must PAUSE, not fail: a photographer whose cable was kic
ked has
    // not lost their wedding.
    let err = outcome.err().expect("must report a run-blocking error");
    assert_eq!(err.code.0, "AURA-IO-1003");
    assert_eq!(err.recovery, Recovery::Retry);

    let files_before = cat.count("photo_file").expect("count");
    assert!(files_before > 0, "work completed before the yank must surv
ive");
    assert_eq!(cat.count_absent().expect("absent"), 0,
        "an unmounted volume must not mark files deleted");
    cat.integrity_check().expect("catalog intact after yank");

    disk.remount().expect("remount");
    let report = resume_import(&cat, disk.path()).expect("resume");
    assert_eq!(cat.count("photo_file").expect("count"), expected_total
());
    assert_eq!(report.files_imported + files_before, expected_total());
    assert_eq!(cat.digest().expect("digest"), reference_digest());
}
```

6. Golden tests and what is allowed to change

```
# tests/golden/manifest.toml
# Every golden output, its owner, and the rule for regenerating it. A g
olden
# with no stated regeneration rule becomes a golden nobody dares to tou
ch.

[[golden]]
```

```
name = "scan-order-small"
path = "tests/golden/scan-order-small.yaml"
owner = "SRC"
regenerate = "cargo insta review"
may_change_when = "the scan order rule itself changes, which requires a
n ADR"

[[golden]]
name = "catalog-schema-v1"
path = "tests/golden/schema-v1.sql"
owner = "SRC"
regenerate = "cargo xtask dump-schema"
may_change_when = "never for v1; a change means a new migration and a n
ew golden"

[[golden]]
name = "error-registry"
path = "tests/golden/errors.snap"
owner = "TLC"
regenerate = "cargo insta review"
may_change_when = "a code is added; existing codes and messages are fro
zen"

[[golden]]
name = "ipc-types"
path = "ui/src/ipc/types.ts"
owner = "SFE"
regenerate = "cargo xtask ipc-types"
may_change_when = "the Rust contract changes, in the same commit"

[[golden]]
name = "import-report-small"
path = "tests/golden/import-report-small.yaml"
owner = "QAL"
regenerate = "cargo insta review"
may_change_when = "the fixture generator changes, which changes the see
d digest too"
```

7. Privacy and security tests

```rust
// tests/security/privacy.rs

/// The poisoned-path test. Import a file whose name is a client's nam
e, then
/// grep every artefact the app produced for that string. This catches
the
/// mistake nobody catches by reading code: a stray `tracing::info!(?pa
th)`.
#[test]
fn no_client_path_appears_in_any_log_or_telemetry_artefact() {
    let secret = "Priyanka-and-Rohan-Villa-Sunset-2026";
    let dir = tempfile::tempdir().expect("tmp");
    let poisoned = dir.path().join(secret).join("IMG_0001.CR2");
    std::fs::create_dir_all(poisoned.parent().expect("parent")).expect
("mkdir");
    std::fs::write(&poisoned, synthetic_bytes()).expect("write");

    let logs = LogCapture::start();              // captures tracing a
t TRACE level
    let cat = fresh_catalog();
    import_folder(&cat, dir.path()).expect("import");
    trigger_an_error_on(&cat, &poisoned);        // force the error pa
th too
    let captured = logs.stop();

    assert!(!captured.contains(secret), "log leaked the client folder n
ame");
    for artefact in [log_file_path(), crash_report_path(), telemetry_qu
eue_path()] {
        if artefact.exists() {
            let body = std::fs::read_to_string(&artefact).unwrap_or_def
ault();
            assert!(!body.contains(secret), "{} leaked the client nam
e", artefact.display());
        }
    }
    // And the catalog DOES contain it, because that is local and neces
sary.
    assert!(cat.contains_path_fragment(secret).expect("query"),
        "the catalog must still record the real path");
```

```
}

#[test]
fn no_network_syscall_occurs_during_a_full_import() { /* NetworkGuard::
deny_all() */ }

#[test]
fn hostile_filenames_never_reach_a_shell() {
    // Assert std::process::Command is not used anywhere in the app cra
tes.
    let hits = grep_workspace("Command::new", &["crates/aura-core", "cr
ates/aura-catalog",
        "crates/aura-ingest", "crates/aura-jobs", "crates/aura-app"]);
    assert!(hits.is_empty(), "process spawning in app code: {hits:?}");
}

#[test]
fn catalog_file_permissions_are_owner_only_on_unix() { /* 0o600 */ }

#[test]
fn consent_defaults_to_denied_after_a_round_trip_through_the_database()
{
    let cat = fresh_catalog();
    let p = cat.create_project("Test").expect("create");
    let reloaded = cat.get_project(p.project_id).expect("reload");
    assert!(!reloaded.consent.cloud_metadata);
    assert!(!reloaded.consent.improve_models);
}

#[test]
fn every_registered_error_code_has_a_runbook_file() {
    for code in every_code_in_registry() {
        let path = format!("docs/runbooks/{code}.md");
        assert!(std::path::Path::new(&path).exists(), "missing runbook:
{path}");
    }
}
```

8. Test hygiene rules, enforced by a test

```
// tests/meta/hygiene.rs

/// Tests about tests. Every one of these has caught a real problem in
a real
/// codebase, which is why they are worth the ten minutes they take to
write.

#[test]
fn no_test_sleeps_to_wait_for_time() {
    let hits = grep_tests("thread::sleep");
    let allowed = ["tests/chaos/"];              // chaos waits on real
processes
    let bad: Vec<_> = hits.into_iter()
        .filter(|h| !allowed.iter().any(|a| h.contains(a))).collect();
    assert!(bad.is_empty(), "use FixedClock instead of sleeping: {ba
d:?}");
}

#[test]
fn no_test_asserts_only_that_something_is_ok() {
    // `assert!(result.is_ok())` with no assertion on the value is a te
st that
    // passes when the function does nothing at all.
    let hits = grep_tests_regex(r"assert!\((\w+)\.is_ok\(\)\);\s*\}");
    assert!(hits.is_empty(), "assert on the value, not just success: {h
its:?}");
}

#[test]
fn every_error_code_is_asserted_somewhere() {
    for code in every_code_in_registry() {
        assert!(grep_tests(&code).len() > 0, "{code} is never asserted
by any test");
    }
}

#[test]
fn no_ignored_test_lacks_a_reason() {
    let hits = grep_tests_regex(r"#\[ignore\](?!\s*=)");
    assert!(hits.is_empty(), "#[ignore] must carry a reason: {hit
```

```
s:?}");
}

#[test]
fn fixture_generation_is_deterministic() {
    let a = generate(Corpus::Small, &tmp_a(), 42).expect("a");
    let b = generate(Corpus::Small, &tmp_b(), 42).expect("b");
    assert_eq!(a.tree_digest(), b.tree_digest(), "same seed must give s
ame bytes");
    let c = generate(Corpus::Small, &tmp_c(), 43).expect("c");
    assert_ne!(a.tree_digest(), c.tree_digest(), "different seed must d
iffer");
}
```

9. Acceptance criteria for T16–T17

- [ ] `cargo xtask fixtures --corpus tiny|small|full|hostile` generates each corpus; same seed gives an identical tree digest on macOS, Windows and Linux
- [ ] The hostile corpus contains all 60 files with a `manifest.toml` declaring the expected outcome of every one
- [ ] Every declared expectation in `manifest.toml` is asserted by a test; no expectation is unused
- [ ] Unit and property lanes finish in under 80 s combined on the reference machine
- [ ] All 14 property tests pass with 256 cases each; the shrink output for a deliberately broken build is pasted to prove shrinking works
- [ ] Integration suite passes against Small plus Hostile in under 4 minutes
- [ ] All 11 chaos scenarios pass, each with its expected error code and a resume under 5 s; the volume-yank and disk-full tests use real mounts, not mocks
- [ ] The poisoned-path privacy test passes, and deliberately adding `tracing::info!(?path)` makes it fail — both directions pasted
- [ ] `no_network_syscall_occurs_during_a_full_import` passes
- [ ] All five golden files exist with an owner and a regeneration rule; `cargo insta test` is clean
- [ ] All five hygiene meta-tests pass
- [ ] Coverage on `aura-core`, `aura-catalog`, `aura-ingest`, `aura-jobs` at or above 85 % line and 75 % branch; report pasted
- [ ] Zero flaky tests across 20 consecutive full runs; the run log is pasted

10. Claude Code prompt for T16–T17

> Act as `QAL` (QA Lead) with `QAIQ` reviewing. Implement T16 and T17 of PHASE-01 from this page.

> Build `cargo xtask fixtures` first, with the SplitMix64 generator exactly as written so fixtures are byte-identical across platforms, all four corpora, and the complete 60-file hostile corpus with its `manifest.toml`. Then write every test in sections 4 through 8. Where a test body is elided with a comment, write it fully.

> Hard constraints: no fixture is committed to git, only generated; no test calls `thread::sleep` to wait for time — use `FixedClock`; every chaos scenario breaks something real (`SIGKILL`, real unmount, real full filesystem), never a mocked failure; every test asserts on a value, not merely on `is_ok()`; every `#[ignore]` carries a reason; every registered error code is asserted by at least one test.

> Prove the suite can actually fail: temporarily break the NFD comparison, the progress monotonicity guard, and the path redaction, and paste the three failing outputs before reverting. A test suite that has never been seen to fail has not been tested.

> Evidence I expect pasted back: `cargo test --workspace` in full; the chaos lane output with all 11 scenarios and their timings; the coverage report; the three deliberate-failure outputs; and the 20-run flake log. Stop after T17.